

midterm

June 2, 2026

0.1 Computer Engineering

0.2 Machine Learning — Regression (Neural Network)

Name: **Robb Ryan P. Calvez** Date: **06/02/2026**

Year & Section: **BSCpE 3-A**

Course: **Computer Engineering - Embedded Systems**

MIDTERM EXAM 1

Import Libraries

```
[9]: import tensorflow as tf
import pandas as pd
import numpy as np
import sklearn
import matplotlib as mpl
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import make_column_transformer
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

print(f'tensorflow: {tf.__version__}\n',
      f'pandas: {pd.__version__}\n',
      f'numpy: {np.__version__}\n',
      f'sklearn: {sklearn.__version__}')
```

```
tensorflow: 2.21.0
pandas: 3.0.2
numpy: 2.4.2
sklearn: 1.8.0
```

Import Datasets

```
[2]: data1 = pd.read_csv("work/ai-1/Assessments/PrelimExam/Problem1Data.csv")
data2 = pd.read_csv("work/ai-1/Assessments/PrelimExam/Problem2Data.csv")
data3 = pd.read_csv("work/ai-1/Assessments/PrelimExam/Problem3Data.csv")
```

data1

```
[2]:   Day ID      Weather Panel Brand  Irradiance (W/m²)  Temp (°C)  Tilt (°)  \
0   D-001  Partly Cloudy    SolarX           720           32           15
1   D-002      Cloudy    VoltMax           580           29           15
2   D-003      Sunny    SolarX           810           34           20
3   D-004    Overcast  GreenRay           430           27           15
4   D-005  Partly Cloudy    VoltMax           650           31           25
5   D-006      Sunny    SolarX           760           33           20
6   D-007      Cloudy  GreenRay           510           28           15
7   D-008      Sunny    SolarX           890           35           25
8   D-009    Overcast  GreenRay           380           26           10
9   D-010  Partly Cloudy    VoltMax           690           31           20
10  D-011      Cloudy    VoltMax           540           30           15
11  D-012      Sunny    SolarX           770           33           25
12  D-013    Overcast  GreenRay           460           27           15
13  D-014      Sunny    SolarX           820           34           20
14  D-015      Hazy    VoltMax           610           30           20
15  D-016    Overcast  GreenRay           350           25           10
16  D-017  Partly Cloudy    SolarX           740           32           25
17  D-018      Cloudy    VoltMax           590           29           15
18  D-019      Sunny    SolarX           870           35           20
19  D-020      Cloudy  GreenRay           470           28           15
20  D-021  Partly Cloudy    VoltMax           700           32           20
21  D-022      Hazy    VoltMax           560           30           25
22  D-023      Sunny    SolarX           830           34           25
23  D-024    Overcast  GreenRay           410           26           10
24  D-025  Partly Cloudy    VoltMax           680           31           20
25  D-026      Hazy    VoltMax           520           29           15
26  D-027      Sunny    SolarX           790           33           25
27  D-028    Overcast  GreenRay           440           27           15
28  D-029  Partly Cloudy    SolarX           750           32           20
29  D-030  Partly Cloudy    VoltMax           630           30           25
30  D-031      Sunny    SolarX           800           33           20
31  D-032      Cloudy    VoltMax           550           29           15
32  D-033    Overcast  GreenRay           390           26           10
33  D-034  Partly Cloudy    SolarX           730           32           25
34  D-035      Hazy    VoltMax           600           30           20
35  D-036      Sunny    VoltMax           780           33           25
36  D-037      Cloudy  GreenRay           490           28           15
37  D-038      Sunny    SolarX           850           34           25
38  D-039    Overcast  GreenRay           420           27           10
39  D-040  Partly Cloudy    VoltMax           670           31           20

      Age (mo)  Dust (g/m²)  Output (kWh)
0             6          1.2          28.4
```

1	18	3.5	21.8
2	3	0.8	32.1
3	36	6.0	14.7
4	12	2.1	24.9
5	8	1.5	30.2
6	24	4.8	18.3
7	2	0.5	34.7
8	48	7.2	11.5
9	14	2.8	26.0
10	30	5.1	19.6
11	5	1.0	31.0
12	42	6.7	14.2
13	10	1.8	31.8
14	20	3.2	23.4
15	54	7.8	10.2
16	7	1.3	29.5
17	22	3.9	21.2
18	4	0.7	33.6
19	38	5.8	15.4
20	16	2.5	26.8
21	26	4.2	20.5
22	6	1.1	32.5
23	50	7.5	12.8
24	13	2.4	25.7
25	28	4.5	18.9
26	9	1.6	30.7
27	44	6.3	13.6
28	11	2.0	28.9
29	19	3.0	24.1
30	4	0.6	32.0
31	26	4.0	20.0
32	52	7.4	11.0
33	5	1.0	29.1
34	25	3.5	22.4
35	11	1.7	30.0
36	32	5.2	17.0
37	3	0.6	33.5
38	46	6.8	13.4
39	15	2.6	25.5

0.2.1 Question 1 - Rooftop Solar Inverter Output

```
[3]: prob1_X = data1.drop(['Output (kWh)'], axis=1)
     prob1_y = data1['Output (kWh)']

     prob1_X, prob1_y
```

[3]: (	Day ID	Weather	Panel Brand	Irradiance (W/m ²)	Temp (°C)	Tilt (°)	\
0	D-001	Partly Cloudy	SolarX	720	32	15	
1	D-002	Cloudy	VoltMax	580	29	15	
2	D-003	Sunny	SolarX	810	34	20	
3	D-004	Overcast	GreenRay	430	27	15	
4	D-005	Partly Cloudy	VoltMax	650	31	25	
5	D-006	Sunny	SolarX	760	33	20	
6	D-007	Cloudy	GreenRay	510	28	15	
7	D-008	Sunny	SolarX	890	35	25	
8	D-009	Overcast	GreenRay	380	26	10	
9	D-010	Partly Cloudy	VoltMax	690	31	20	
10	D-011	Cloudy	VoltMax	540	30	15	
11	D-012	Sunny	SolarX	770	33	25	
12	D-013	Overcast	GreenRay	460	27	15	
13	D-014	Sunny	SolarX	820	34	20	
14	D-015	Hazy	VoltMax	610	30	20	
15	D-016	Overcast	GreenRay	350	25	10	
16	D-017	Partly Cloudy	SolarX	740	32	25	
17	D-018	Cloudy	VoltMax	590	29	15	
18	D-019	Sunny	SolarX	870	35	20	
19	D-020	Cloudy	GreenRay	470	28	15	
20	D-021	Partly Cloudy	VoltMax	700	32	20	
21	D-022	Hazy	VoltMax	560	30	25	
22	D-023	Sunny	SolarX	830	34	25	
23	D-024	Overcast	GreenRay	410	26	10	
24	D-025	Partly Cloudy	VoltMax	680	31	20	
25	D-026	Hazy	VoltMax	520	29	15	
26	D-027	Sunny	SolarX	790	33	25	
27	D-028	Overcast	GreenRay	440	27	15	
28	D-029	Partly Cloudy	SolarX	750	32	20	
29	D-030	Partly Cloudy	VoltMax	630	30	25	
30	D-031	Sunny	SolarX	800	33	20	
31	D-032	Cloudy	VoltMax	550	29	15	
32	D-033	Overcast	GreenRay	390	26	10	
33	D-034	Partly Cloudy	SolarX	730	32	25	
34	D-035	Hazy	VoltMax	600	30	20	
35	D-036	Sunny	VoltMax	780	33	25	
36	D-037	Cloudy	GreenRay	490	28	15	
37	D-038	Sunny	SolarX	850	34	25	
38	D-039	Overcast	GreenRay	420	27	10	
39	D-040	Partly Cloudy	VoltMax	670	31	20	
	Age (mo)	Dust (g/m ²)					
0	6	1.2					
1	18	3.5					
2	3	0.8					
3	36	6.0					

4	12	2.1
5	8	1.5
6	24	4.8
7	2	0.5
8	48	7.2
9	14	2.8
10	30	5.1
11	5	1.0
12	42	6.7
13	10	1.8
14	20	3.2
15	54	7.8
16	7	1.3
17	22	3.9
18	4	0.7
19	38	5.8
20	16	2.5
21	26	4.2
22	6	1.1
23	50	7.5
24	13	2.4
25	28	4.5
26	9	1.6
27	44	6.3
28	11	2.0
29	19	3.0
30	4	0.6
31	26	4.0
32	52	7.4
33	5	1.0
34	25	3.5
35	11	1.7
36	32	5.2
37	3	0.6
38	46	6.8
39	15	2.6 ,
0	28.4	
1	21.8	
2	32.1	
3	14.7	
4	24.9	
5	30.2	
6	18.3	
7	34.7	
8	11.5	
9	26.0	
10	19.6	

```

11    31.0
12    14.2
13    31.8
14    23.4
15    10.2
16    29.5
17    21.2
18    33.6
19    15.4
20    26.8
21    20.5
22    32.5
23    12.8
24    25.7
25    18.9
26    30.7
27    13.6
28    28.9
29    24.1
30    32.0
31    20.0
32    11.0
33    29.1
34    22.4
35    30.0
36    17.0
37    33.5
38    13.4
39    25.5
Name: Output (kWh), dtype: float64)

```

Split dataset into Train/Test sets

```

[4]: prob1_X_train, prob1_X_test, prob1_y_train, prob1_y_test = \
    ↪ train_test_split(prob1_X, prob1_y,
    ↪ shuffle=False,
    ↪ test_size=0.25
    )
prob1_X_train

```

```

[4]:   Day ID      Weather Panel Brand  Irradiance (W/m²)  Temp (°C)  Tilt (°)  \
0   D-001  Partly Cloudy      SolarX           720         32      15
1   D-002    Cloudy      VoltMax           580         29      15
2   D-003    Sunny      SolarX           810         34      20
3   D-004  Overcast  GreenRay           430         27      15

```

4	D-005	Partly Cloudy	VoltMax	650	31	25
5	D-006	Sunny	SolarX	760	33	20
6	D-007	Cloudy	GreenRay	510	28	15
7	D-008	Sunny	SolarX	890	35	25
8	D-009	Overcast	GreenRay	380	26	10
9	D-010	Partly Cloudy	VoltMax	690	31	20
10	D-011	Cloudy	VoltMax	540	30	15
11	D-012	Sunny	SolarX	770	33	25
12	D-013	Overcast	GreenRay	460	27	15
13	D-014	Sunny	SolarX	820	34	20
14	D-015	Hazy	VoltMax	610	30	20
15	D-016	Overcast	GreenRay	350	25	10
16	D-017	Partly Cloudy	SolarX	740	32	25
17	D-018	Cloudy	VoltMax	590	29	15
18	D-019	Sunny	SolarX	870	35	20
19	D-020	Cloudy	GreenRay	470	28	15
20	D-021	Partly Cloudy	VoltMax	700	32	20
21	D-022	Hazy	VoltMax	560	30	25
22	D-023	Sunny	SolarX	830	34	25
23	D-024	Overcast	GreenRay	410	26	10
24	D-025	Partly Cloudy	VoltMax	680	31	20
25	D-026	Hazy	VoltMax	520	29	15
26	D-027	Sunny	SolarX	790	33	25
27	D-028	Overcast	GreenRay	440	27	15
28	D-029	Partly Cloudy	SolarX	750	32	20
29	D-030	Partly Cloudy	VoltMax	630	30	25

	Age (mo)	Dust (g/m ²)
0	6	1.2
1	18	3.5
2	3	0.8
3	36	6.0
4	12	2.1
5	8	1.5
6	24	4.8
7	2	0.5
8	48	7.2
9	14	2.8
10	30	5.1
11	5	1.0
12	42	6.7
13	10	1.8
14	20	3.2
15	54	7.8
16	7	1.3
17	22	3.9
18	4	0.7

19	38	5.8
20	16	2.5
21	26	4.2
22	6	1.1
23	50	7.5
24	13	2.4
25	28	4.5
26	9	1.6
27	44	6.3
28	11	2.0
29	19	3.0

Encode and Scale dataset

```
[5]: ct = make_column_transformer(
    (OneHotEncoder(), ['Weather', 'Panel Brand']), #OneHotEncoder() on
    ↪categorical data
    (StandardScaler(), ['Irradiance (W/m²)', 'Temp (°C)', 'Tilt (°)', 'Age_
    ↪(mo)', 'Dust (g/m²)']) #StandardScaler() as the z-score scaler
)

ct.fit(prob1_X_train)

prob1_X_train_scaled = ct.transform(prob1_X_train)
prob1_X_test_scaled = ct.transform(prob1_X_test)
```

```
[6]: prob1_X_train_scaled[0]
```

```
[6]: array([ 0.          ,  0.          ,  0.          ,  1.          ,  0.          ,
            0.          ,  1.          ,  0.          ,  0.57597128,  0.56171733,
            -0.76088591, -0.97454472, -1.00399945])
```

Create Model

```
[ ]: tf.random.set_seed(42)

prob1_model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(13,)),
    tf.keras.layers.Dense(16, activation='relu'),
    tf.keras.layers.Dense(8, activation='relu'),
    tf.keras.layers.Dense(1, activation='linear')
])

prob1_model.compile(loss='mse',
                    optimizer=tf.keras.optimizers.Adam(learning_rate=0.
    ↪001)
                    )
```



```

history1 = prob1_model.fit(prob1_X_train_scaled, prob1_y_train,
                           epochs=200,
                           verbose=0,
                           batch_size=8
                           )

```

```

I0000 00:00:1780334561.802823    95748 gpu_device.cc:2043] Created device
/job:localhost/replica:0/task:0/device:GPU:0 with 3536 MB memory:  -> device: 0,
name: NVIDIA GeForce RTX 4050 Laptop GPU, pci bus id: 0000:01:00.0, compute
capability: 8.9
I0000 00:00:1780334564.012517    95930 service.cc:153] XLA service 0x702ac80317d0
initialized for platform CUDA (this does not guarantee that XLA will be used).
Devices:
I0000 00:00:1780334564.012577    95930 service.cc:161]   StreamExecutor [0]:
NVIDIA GeForce RTX 4050 Laptop GPU, Compute Capability 8.9 (Driver: 13.2.0;
Runtime: 12.3.0; Toolkit: 12.5.0; DNN: 9.20.0)
I0000 00:00:1780334564.040745    95930 dump_mlir_util.cc:269] disabling MLIR
crash reproducer, set env var `MLIR_CRASH_REPRODUCER_DIRECTORY` to enable.
I0000 00:00:1780334564.189436    95930 cuda_dnn.cc:461] Loaded cuDNN version
92000
I0000 00:00:1780334564.202530    95930 dot_merger.cc:481] Merging Dots in
computation: a_inference_one_step_on_data_1160__.7
I0000 00:00:1780334565.405852    95930 device_compiler.h:208] Compiled cluster
using XLA! This line is logged at most once for the lifetime of the process.
I0000 00:00:1780334565.555173    95931 dot_merger.cc:481] Merging Dots in
computation: a_inference_one_step_on_data_1160__.7

```

Evaluate Model

```

[10]: prob1_y_pred = prob1_model.predict(prob1_X_test_scaled)

```

```

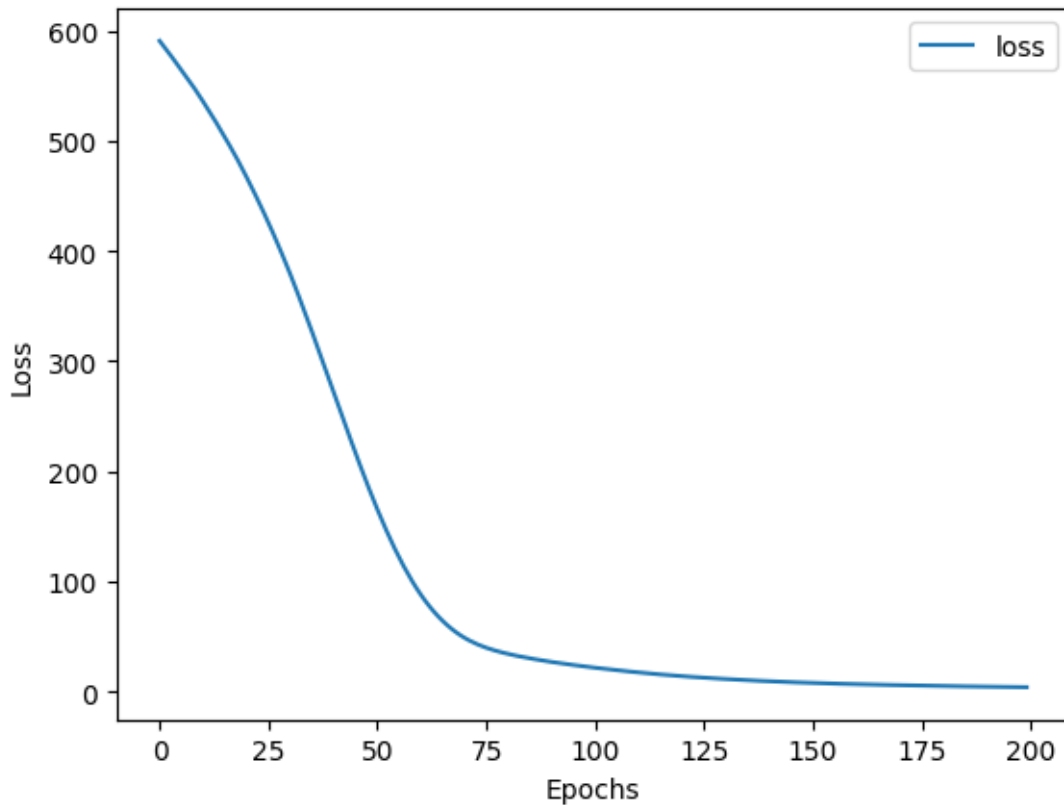
1/1          1s 832ms/step

```

```

[8]: pd.DataFrame(history1.history).plot()
     mpl.pyplot.xlabel("Epochs")
     mpl.pyplot.ylabel("Loss");

```



```
[12]: mae = mean_absolute_error(prob1_y_test, prob1_y_pred)
      rmse = np.sqrt(mean_squared_error(prob1_y_test, prob1_y_pred))
      r2 = r2_score(prob1_y_test, prob1_y_pred)

      print(f"MAE: {mae:.4f}")
      print(f"RMSE: {rmse:.4f}")
      print(f"R²: {r2:.4f}")
```

```
MAE: 1.4654
RMSE: 1.7915
R²: 0.9431
```

```
[14]: comparison = pd.DataFrame({
      'Actual': prob1_y_test,
      'Predicted': prob1_y_pred.flatten(),
      'Error': prob1_y_test - prob1_y_pred.flatten()
    })
      print(comparison)
```

	Actual	Predicted	Error
30	32.0	31.574574	0.425426
31	20.0	19.034611	0.965389

```

32    11.0  14.213167 -3.213167
33    29.1  31.593945 -2.493945
34    22.4  21.259996  1.140004
35    30.0  31.099356 -1.099356
36    17.0  14.438525  2.561475
37    33.5  35.789692 -2.289692
38    13.4  13.070896  0.329104
39    25.5  25.636276 -0.136276

```

0.2.2 Question 2 - Electric Vehicle Remaining Range

```
[23]: data2
```

```

[23]:   Trip ID   Mode Road Type  SoC (%)  Temp (°C)  AC (kW)  Elev (m)  \
0    T-001    Eco    Urban      80      28      1.2      50
1    T-002   Sport  Highway     50      35      2.5     120
2    T-003    Eco    Urban     95      24      0.8      20
3    T-004   Sport  Mountain    30      33      2.0     200
4    T-005  Normal   Mixed     65      30      1.5      80
5    T-006    Eco    Urban     85      26      1.0      40
6    T-007   Sport  Mountain    45      34      2.2     150
7    T-008    Eco    Urban     90      25      0.9      30
8    T-009   Sport  Mountain    25      36      2.8     220
9    T-010  Normal   Mixed     70      29      1.4      70
10   T-011   Sport  Highway     55      32      1.8     110
11   T-012    Eco    Urban     88      27      1.1      45
12   T-013   Sport  Mountain    40      33      2.3     170
13   T-014  Normal   Mixed     75      28      1.3      65
14   T-015  Normal   Highway    60      31      1.7     100
15   T-016   Sport  Mountain    35      35      2.6     190
16   T-017    Eco    Urban     92      26      1.0      35
17   T-018  Normal   Mixed     58      30      1.6      90
18   T-019  Normal   Mixed     78      29      1.3      60
19   T-020   Sport  Mountain    48      34      2.4     160
20   T-021    Eco    Urban     82      27      1.1      55
21   T-022   Sport  Mountain    38      33      2.5     180
22   T-023  Normal   Mixed     68      30      1.5      75
23   T-024   Sport  Mountain    28      36      2.7     210
24   T-025    Eco    Urban     87      26      1.0      38
25   T-026   Sport  Highway     52      32      1.9     130
26   T-027  Normal   Mixed     72      29      1.4      68
27   T-028   Sport  Mountain    43      34      2.3     165
28   T-029  Normal   Highway    63      31      1.7      95
29   T-030   Sport  Mountain    33      35      2.6     195
30   T-031    Eco    Urban     83      27      1.1      42
31   T-032   Sport  Mountain    32      35      2.7     205
32   T-033  Normal   Mixed     67      30      1.5      78

```

33	T-034	Eco	Urban	91	25	0.9	28
34	T-035	Sport	Highway	53	32	1.9	125
35	T-036	Normal	Highway	62	31	1.7	98
36	T-037	Sport	Mountain	37	34	2.4	175
37	T-038	Eco	Urban	89	26	1.0	36
38	T-039	Normal	Mixed	74	28	1.3	67
39	T-040	Sport	Mountain	27	36	2.8	215

	Speed (km/h)	Range (km)
0	55	232
1	70	118
2	50	305
3	80	58
4	60	175
5	55	258
6	75	92
7	50	285
8	85	42
9	60	198
10	70	130
11	55	265
12	75	78
13	60	215
14	65	158
15	80	65
16	50	295
17	65	152
18	58	222
19	75	88
20	55	244
21	78	70
22	62	188
23	82	48
24	52	272
25	72	105
26	60	205
27	76	82
28	65	165
29	80	60
30	53	245
31	81	55
32	61	182
33	51	290
34	71	112
35	65	160
36	77	72
37	53	278

```

38         60         212
39         83         45

```

```

[24]: prob2_X = data2.drop(['Range (km)'], axis=1)
      prob2_y = data2['Range (km)']

      prob2_X, prob2_y

```

```

[24]: (   Trip ID   Mode Road Type  SoC (%)  Temp (°C)  AC (kW)  Elev (m)  \
0     T-001    Eco    Urban        80         28      1.2      50
1     T-002   Sport  Highway        50         35      2.5     120
2     T-003    Eco    Urban        95         24      0.8      20
3     T-004   Sport  Mountain        30         33      2.0     200
4     T-005  Normal   Mixed        65         30      1.5      80
5     T-006    Eco    Urban        85         26      1.0      40
6     T-007   Sport  Mountain        45         34      2.2     150
7     T-008    Eco    Urban        90         25      0.9      30
8     T-009   Sport  Mountain        25         36      2.8     220
9     T-010  Normal   Mixed        70         29      1.4      70
10    T-011   Sport  Highway        55         32      1.8     110
11    T-012    Eco    Urban        88         27      1.1      45
12    T-013   Sport  Mountain        40         33      2.3     170
13    T-014  Normal   Mixed        75         28      1.3      65
14    T-015  Normal  Highway        60         31      1.7     100
15    T-016   Sport  Mountain        35         35      2.6     190
16    T-017    Eco    Urban        92         26      1.0      35
17    T-018  Normal   Mixed        58         30      1.6      90
18    T-019  Normal   Mixed        78         29      1.3      60
19    T-020   Sport  Mountain        48         34      2.4     160
20    T-021    Eco    Urban        82         27      1.1      55
21    T-022   Sport  Mountain        38         33      2.5     180
22    T-023  Normal   Mixed        68         30      1.5      75
23    T-024   Sport  Mountain        28         36      2.7     210
24    T-025    Eco    Urban        87         26      1.0      38
25    T-026   Sport  Highway        52         32      1.9     130
26    T-027  Normal   Mixed        72         29      1.4      68
27    T-028   Sport  Mountain        43         34      2.3     165
28    T-029  Normal  Highway        63         31      1.7      95
29    T-030   Sport  Mountain        33         35      2.6     195
30    T-031    Eco    Urban        83         27      1.1      42
31    T-032   Sport  Mountain        32         35      2.7     205
32    T-033  Normal   Mixed        67         30      1.5      78
33    T-034    Eco    Urban        91         25      0.9      28
34    T-035   Sport  Highway        53         32      1.9     125
35    T-036  Normal  Highway        62         31      1.7      98
36    T-037   Sport  Mountain        37         34      2.4     175
37    T-038    Eco    Urban        89         26      1.0      36

```

38	T-039	Normal	Mixed	74	28	1.3	67
39	T-040	Sport	Mountain	27	36	2.8	215

Speed (km/h)

0	55
1	70
2	50
3	80
4	60
5	55
6	75
7	50
8	85
9	60
10	70
11	55
12	75
13	60
14	65
15	80
16	50
17	65
18	58
19	75
20	55
21	78
22	62
23	82
24	52
25	72
26	60
27	76
28	65
29	80
30	53
31	81
32	61
33	51
34	71
35	65
36	77
37	53
38	60
39	83 ,
0	232
1	118
2	305

3	58
4	175
5	258
6	92
7	285
8	42
9	198
10	130
11	265
12	78
13	215
14	158
15	65
16	295
17	152
18	222
19	88
20	244
21	70
22	188
23	48
24	272
25	105
26	205
27	82
28	165
29	60
30	245
31	55
32	182
33	290
34	112
35	160
36	72
37	278
38	212
39	45

Name: Range (km), dtype: int64)

Split dataset into Train/Test sets

```
[25]: prob2_X_train, prob2_X_test, prob2_y_train, prob2_y_test =  
      ↪ train_test_split(prob2_X, prob2_y,  
      ↪ shuffle=False,  
      ↪ test_size=0.25
```

prob2_X_train

```
[25]: Trip ID    Mode Road Type SoC (%) Temp (°C) AC (kW) Elev (m) \
0      T-001    Eco    Urban      80      28      1.2      50
1      T-002  Sport Highway      50      35      2.5     120
2      T-003    Eco    Urban      95      24      0.8      20
3      T-004  Sport Mountain      30      33      2.0     200
4      T-005 Normal   Mixed      65      30      1.5      80
5      T-006    Eco    Urban      85      26      1.0      40
6      T-007  Sport Mountain      45      34      2.2     150
7      T-008    Eco    Urban      90      25      0.9      30
8      T-009  Sport Mountain      25      36      2.8     220
9      T-010 Normal   Mixed      70      29      1.4      70
10     T-011  Sport Highway      55      32      1.8     110
11     T-012    Eco    Urban      88      27      1.1      45
12     T-013  Sport Mountain      40      33      2.3     170
13     T-014 Normal   Mixed      75      28      1.3      65
14     T-015 Normal   Highway      60      31      1.7     100
15     T-016  Sport Mountain      35      35      2.6     190
16     T-017    Eco    Urban      92      26      1.0      35
17     T-018 Normal   Mixed      58      30      1.6      90
18     T-019 Normal   Mixed      78      29      1.3      60
19     T-020  Sport Mountain      48      34      2.4     160
20     T-021    Eco    Urban      82      27      1.1      55
21     T-022  Sport Mountain      38      33      2.5     180
22     T-023 Normal   Mixed      68      30      1.5      75
23     T-024  Sport Mountain      28      36      2.7     210
24     T-025    Eco    Urban      87      26      1.0      38
25     T-026  Sport Highway      52      32      1.9     130
26     T-027 Normal   Mixed      72      29      1.4      68
27     T-028  Sport Mountain      43      34      2.3     165
28     T-029 Normal   Highway      63      31      1.7      95
29     T-030  Sport Mountain      33      35      2.6     195
```

```
Speed (km/h)
0           55
1           70
2           50
3           80
4           60
5           55
6           75
7           50
8           85
9           60
10          70
```


11	55
12	75
13	60
14	65
15	80
16	50
17	65
18	58
19	75
20	55
21	78
22	62
23	82
24	52
25	72
26	60
27	76
28	65
29	80

Encode and Scale dataset

```
[26]: ct = make_column_transformer(
    (OneHotEncoder(), ['Mode', 'Road Type']), #OneHotEncoder() on categorical
    ↪data
    (StandardScaler(), ['SoC (%)', 'Temp (°C)', 'AC (kW)', 'Elev (m)', 'Speed_
    ↪(km/h)']) #StandardScaler() as the z-score scaler
)

ct.fit(prob2_X_train)

prob2_X_train_scaled = ct.transform(prob2_X_train)
prob2_X_test_scaled = ct.transform(prob2_X_test)
```

```
[27]: prob2_X_train_scaled[0]
```

```
[27]: array([ 1.          ,  0.          ,  0.          ,  0.          ,  0.          ,
            0.          ,  1.          ,  0.90339726, -0.75139069, -0.88030731,
            -0.93819755, -1.01253967])
```

Create Model

```
[28]: tf.random.set_seed(42)

prob2_model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(12,)),
    tf.keras.layers.Dense(16, activation='relu'),
    tf.keras.layers.Dense(8, activation='relu'),
```

```

    tf.keras.layers.Dense(1, activation='linear')
])

prob2_model.compile(loss='mse',
                    optimizer=tf.keras.optimizers.Adam(learning_rate=0.
↪001)
                    )

history2 = prob2_model.fit(prob2_X_train_scaled, prob2_y_train,
                           epochs=200,
                           verbose=0,
                           batch_size=8
                           )

```

```

I0000 00:00:1780338006.167484    95930 dot_merger.cc:481] Merging Dots in
computation: a_inference_one_step_on_data_6261__.7
I0000 00:00:1780338007.355402    95929 dot_merger.cc:481] Merging Dots in
computation: a_inference_one_step_on_data_6261__.7

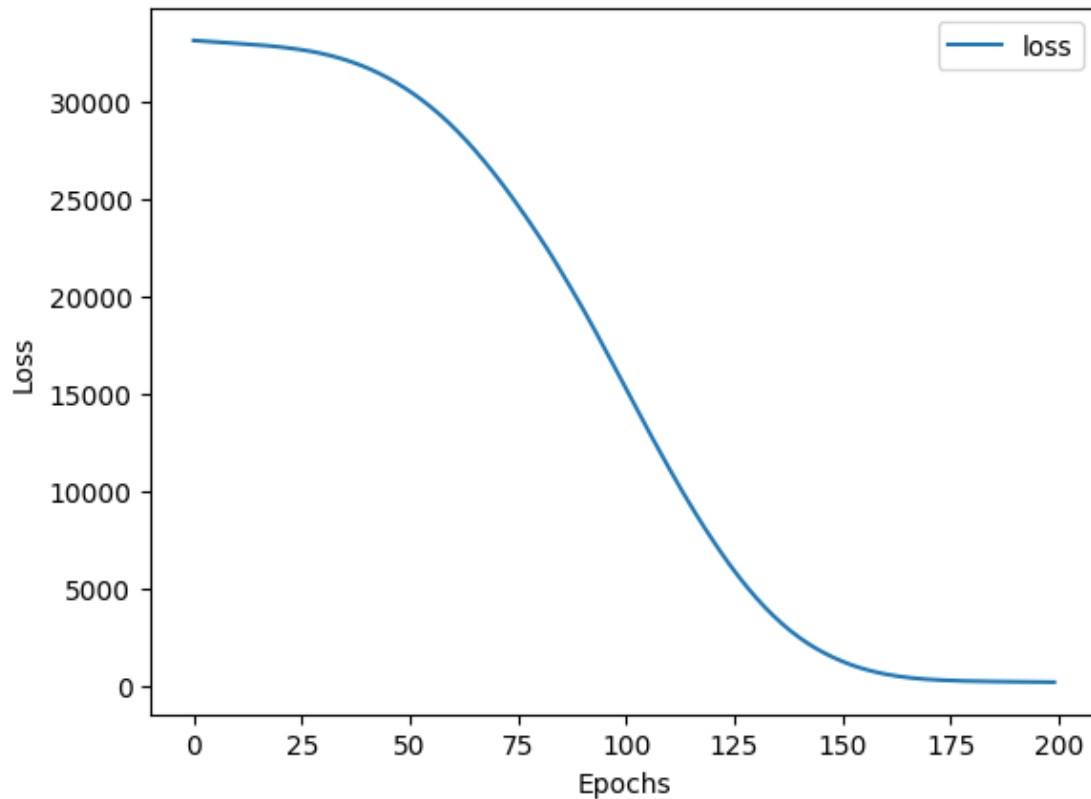
```

Evaluate Model

```
[29]: prob2_y_pred = prob2_model.predict(prob2_X_test_scaled)
```

```
1/1          0s 352ms/step
```

```
[30]: pd.DataFrame(history2.history).plot()
      mpl.pyplot.xlabel("Epochs")
      mpl.pyplot.ylabel("Loss");
```



```
[31]: mae = mean_absolute_error(prob2_y_test, prob2_y_pred)
      rmse = np.sqrt(mean_squared_error(prob2_y_test, prob2_y_pred))
      r2 = r2_score(prob2_y_test, prob2_y_pred)

      print(f"MAE: {mae:.4f}")
      print(f"RMSE: {rmse:.4f}")
      print(f"R²: {r2:.4f}")
```

```
MAE: 8.4825
RMSE: 11.7757
R²: 0.9816
```

```
[32]: comparison = pd.DataFrame({
      'Actual': prob2_y_test,
      'Predicted': prob2_y_pred.flatten(),
      'Error': prob2_y_test - prob2_y_pred.flatten()
    })
      print(comparison)
```

	Actual	Predicted	Error
30	245	251.713760	-6.713760
31	55	70.822258	-15.822258

```

32      182  180.410110   1.589890
33      290  297.552826  -7.552826
34      112  107.533760   4.466240
35      160  152.580383   7.419617
36       72   65.931953   6.068047
37      278  275.139893   2.860107
38      212  214.220078  -2.220078
39       45   75.112373 -30.112373

```

0.2.3 Question 3 - Data-Center Server Response Latency

```
[38]: data3
```

```

[38]: Snap ID Request Type      Region CPU (%) Conns  Cache (%) BW (Mbps) \
0    S-001  Read-Heavy    US-East   45    320      92      850
1    S-002    Mixed    Asia-Pacific  78   1100      65      320
2    S-003  Read-Heavy    EU-West   30    180      95      940
3    S-004  Write-Heavy  Latin-America  88   1500      58      210
4    S-005    Mixed    US-East   62    720      81      600
5    S-006  Read-Heavy    EU-West   52    420      88      780
6    S-007  Write-Heavy  Asia-Pacific  82   1280      60      280
7    S-008  Read-Heavy    US-East   38    240      93      880
8    S-009  Write-Heavy  Latin-America  92   1620      55      180
9    S-010    Mixed    EU-West   68    850      78      540
10   S-011  Read-Heavy    US-East   48    380      89      820
11   S-012    Mixed    Asia-Pacific  75   1020      70      380
12   S-013  Read-Heavy    EU-West   34    210      94      920
13   S-014  Write-Heavy  Latin-America  85   1380      62      250
14   S-015    Mixed    US-East   58    560      84      690
15   S-016  Write-Heavy  Latin-America  95   1750      52      160
16   S-017  Read-Heavy    EU-West   42    290      91      800
17   S-018    Mixed    Asia-Pacific  72    920      74      460
18   S-019  Read-Heavy    US-East   50    460      87      740
19   S-020  Write-Heavy  Latin-America  80   1180      64      300
20   S-021  Read-Heavy    EU-West   36    220      94      900
21   S-022    Mixed    Asia-Pacific  65    780      79      580
22   S-023  Write-Heavy  Latin-America  90   1540      56      200
23   S-024    Mixed    US-East   55    510      85      660
24   S-025    Mixed    Asia-Pacific  70    880      76      500
25   S-026  Read-Heavy    EU-West   45    350      90      810
26   S-027  Write-Heavy  Latin-America  86   1440      59      230
27   S-028    Mixed    US-East   60    660      82      620
28   S-029  Read-Heavy    EU-West   32    200      96      930
29   S-030    Mixed    Asia-Pacific  77   1080      68      360
30   S-031  Read-Heavy    US-East   40    280      92      830
31   S-032  Write-Heavy  Latin-America  87   1460      60      220
32   S-033    Mixed    Asia-Pacific  73    970      72      420

```

33	S-034	Read-Heavy	EU-West	33	200	95	910
34	S-035	Mixed	US-East	57	540	84	680
35	S-036	Write-Heavy	Latin-America	93	1680	54	170
36	S-037	Read-Heavy	EU-West	47	360	90	830
37	S-038	Mixed	Asia-Pacific	66	810	78	560
38	S-039	Write-Heavy	US-East	84	1320	61	260
39	S-040	Mixed	EU-West	53	470	86	700

	Queue	Latency (ms)
0	12	38
1	85	142
2	5	22
3	130	198
4	40	76
5	18	48
6	110	175
7	8	28
8	145	215
9	52	92
10	15	42
11	78	128
12	6	25
13	120	188
14	30	65
15	158	232
16	11	35
17	65	108
18	22	52
19	95	162
20	7	26
21	45	84
22	138	208
23	26	58
24	58	98
25	14	40
26	125	195
27	36	72
28	4	23
29	82	135
30	10	32
31	132	200
32	70	118
33	5	24
34	28	62
35	152	225
36	13	41
37	48	88

```

38      115          180
39      24           55

```

```

[39]: prob3_X = data3.drop(['Latency (ms)'], axis=1)
      prob3_y = data3['Latency (ms)']

      prob3_X, prob3_y

```

```

[39]: (  Snap ID Request Type      Region CPU (%) Conns  Cache (%) BW (Mbps) \
0    S-001  Read-Heavy    US-East    45    320      92    850
1    S-002    Mixed    Asia-Pacific    78   1100      65    320
2    S-003  Read-Heavy    EU-West    30    180      95    940
3    S-004  Write-Heavy  Latin-America    88   1500      58    210
4    S-005    Mixed    US-East    62    720      81    600
5    S-006  Read-Heavy    EU-West    52    420      88    780
6    S-007  Write-Heavy  Asia-Pacific    82   1280      60    280
7    S-008  Read-Heavy    US-East    38    240      93    880
8    S-009  Write-Heavy  Latin-America    92   1620      55    180
9    S-010    Mixed    EU-West    68    850      78    540
10   S-011  Read-Heavy    US-East    48    380      89    820
11   S-012    Mixed    Asia-Pacific    75   1020      70    380
12   S-013  Read-Heavy    EU-West    34    210      94    920
13   S-014  Write-Heavy  Latin-America    85   1380      62    250
14   S-015    Mixed    US-East    58    560      84    690
15   S-016  Write-Heavy  Latin-America    95   1750      52    160
16   S-017  Read-Heavy    EU-West    42    290      91    800
17   S-018    Mixed    Asia-Pacific    72    920      74    460
18   S-019  Read-Heavy    US-East    50    460      87    740
19   S-020  Write-Heavy  Latin-America    80   1180      64    300
20   S-021  Read-Heavy    EU-West    36    220      94    900
21   S-022    Mixed    Asia-Pacific    65    780      79    580
22   S-023  Write-Heavy  Latin-America    90   1540      56    200
23   S-024    Mixed    US-East    55    510      85    660
24   S-025    Mixed    Asia-Pacific    70    880      76    500
25   S-026  Read-Heavy    EU-West    45    350      90    810
26   S-027  Write-Heavy  Latin-America    86   1440      59    230
27   S-028    Mixed    US-East    60    660      82    620
28   S-029  Read-Heavy    EU-West    32    200      96    930
29   S-030    Mixed    Asia-Pacific    77   1080      68    360
30   S-031  Read-Heavy    US-East    40    280      92    830
31   S-032  Write-Heavy  Latin-America    87   1460      60    220
32   S-033    Mixed    Asia-Pacific    73    970      72    420
33   S-034  Read-Heavy    EU-West    33    200      95    910
34   S-035    Mixed    US-East    57    540      84    680
35   S-036  Write-Heavy  Latin-America    93   1680      54    170
36   S-037  Read-Heavy    EU-West    47    360      90    830
37   S-038    Mixed    Asia-Pacific    66    810      78    560

```

38	S-039	Write-Heavy	US-East	84	1320	61	260
39	S-040	Mixed	EU-West	53	470	86	700

	Queue
0	12
1	85
2	5
3	130
4	40
5	18
6	110
7	8
8	145
9	52
10	15
11	78
12	6
13	120
14	30
15	158
16	11
17	65
18	22
19	95
20	7
21	45
22	138
23	26
24	58
25	14
26	125
27	36
28	4
29	82
30	10
31	132
32	70
33	5
34	28
35	152
36	13
37	48
38	115
39	24 ,
0	38
1	142
2	22

3	198
4	76
5	48
6	175
7	28
8	215
9	92
10	42
11	128
12	25
13	188
14	65
15	232
16	35
17	108
18	52
19	162
20	26
21	84
22	208
23	58
24	98
25	40
26	195
27	72
28	23
29	135
30	32
31	200
32	118
33	24
34	62
35	225
36	41
37	88
38	180
39	55

Name: Latency (ms), dtype: int64)

Split dataset into Train/Test sets

```
[40]: prob3_X_train, prob3_X_test, prob3_y_train, prob3_y_test =  
      ↪ train_test_split(prob3_X, prob3_y,  
      ↪ shuffle=False,  
      ↪ test_size=0.25
```


prob3_X_train

```
[40]: Snap ID Request Type      Region CPU (%) Conns Cache (%) BW (Mbps) \
0    S-001  Read-Heavy      US-East   45   320    92    850
1    S-002    Mixed      Asia-Pacific  78  1100    65    320
2    S-003  Read-Heavy      EU-West   30   180    95    940
3    S-004  Write-Heavy Latin-America  88  1500    58    210
4    S-005    Mixed      US-East   62   720    81    600
5    S-006  Read-Heavy      EU-West   52   420    88    780
6    S-007  Write-Heavy      Asia-Pacific  82  1280    60    280
7    S-008  Read-Heavy      US-East   38   240    93    880
8    S-009  Write-Heavy Latin-America  92  1620    55    180
9    S-010    Mixed      EU-West   68   850    78    540
10   S-011  Read-Heavy      US-East   48   380    89    820
11   S-012    Mixed      Asia-Pacific  75  1020    70    380
12   S-013  Read-Heavy      EU-West   34   210    94    920
13   S-014  Write-Heavy Latin-America  85  1380    62    250
14   S-015    Mixed      US-East   58   560    84    690
15   S-016  Write-Heavy Latin-America  95  1750    52    160
16   S-017  Read-Heavy      EU-West   42   290    91    800
17   S-018    Mixed      Asia-Pacific  72   920    74    460
18   S-019  Read-Heavy      US-East   50   460    87    740
19   S-020  Write-Heavy Latin-America  80  1180    64    300
20   S-021  Read-Heavy      EU-West   36   220    94    900
21   S-022    Mixed      Asia-Pacific  65   780    79    580
22   S-023  Write-Heavy Latin-America  90  1540    56    200
23   S-024    Mixed      US-East   55   510    85    660
24   S-025    Mixed      Asia-Pacific  70   880    76    500
25   S-026  Read-Heavy      EU-West   45   350    90    810
26   S-027  Write-Heavy Latin-America  86  1440    59    230
27   S-028    Mixed      US-East   60   660    82    620
28   S-029  Read-Heavy      EU-West   32   200    96    930
29   S-030    Mixed      Asia-Pacific  77  1080    68    360
```

```
Queue
0    12
1    85
2     5
3   130
4    40
5    18
6   110
7     8
8   145
9    52
10   15
```

```

11      78
12      6
13     120
14      30
15     158
16      11
17      65
18      22
19      95
20       7
21      45
22     138
23      26
24      58
25      14
26     125
27      36
28       4
29      82

```

Encode and Scale dataset

```

[41]: ct = make_column_transformer(
        (OneHotEncoder(), ['Request Type', 'Region']), #OneHotEncoder() on
        ↪ categorical data
        (StandardScaler(), ['CPU (%)', 'Conns', 'Cache (%)', 'BW (Mbps)', 'Queue']))
        ↪ #StandardScaler() as the z-score scaler
    )

    ct.fit(prob3_X_train)

    prob3_X_train_scaled = ct.transform(prob3_X_train)
    prob3_X_test_scaled = ct.transform(prob3_X_test)

```

```

[42]: prob3_X_train_scaled[0]

```

```

[42]: array([ 0.          ,  1.          ,  0.          ,  0.          ,  0.          ,
            0.          ,  1.          , -0.91720613, -0.99550607,  1.05896952,
            1.09385136, -0.94776554])

```

Create Model

```

[43]: tf.random.set_seed(42)

    prob3_model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(12,)),
        tf.keras.layers.Dense(16, activation='relu'),
        tf.keras.layers.Dense(8, activation='relu'),

```

```

    tf.keras.layers.Dense(1, activation='linear')
])

prob3_model.compile(loss='mse',
                    optimizer=tf.keras.optimizers.Adam(learning_rate=0.
↪001)
                    )

history3 = prob3_model.fit(prob3_X_train_scaled, prob3_y_train,
                           epochs=200,
                           verbose=0,
                           batch_size=8
                           )

```

```

I0000 00:00:1780338380.708315    95929 dot_merger.cc:481] Merging Dots in
computation: a_inference_one_step_on_data_11362__.7
I0000 00:00:1780338381.351173    95931 dot_merger.cc:481] Merging Dots in
computation: a_inference_one_step_on_data_11362__.7

```

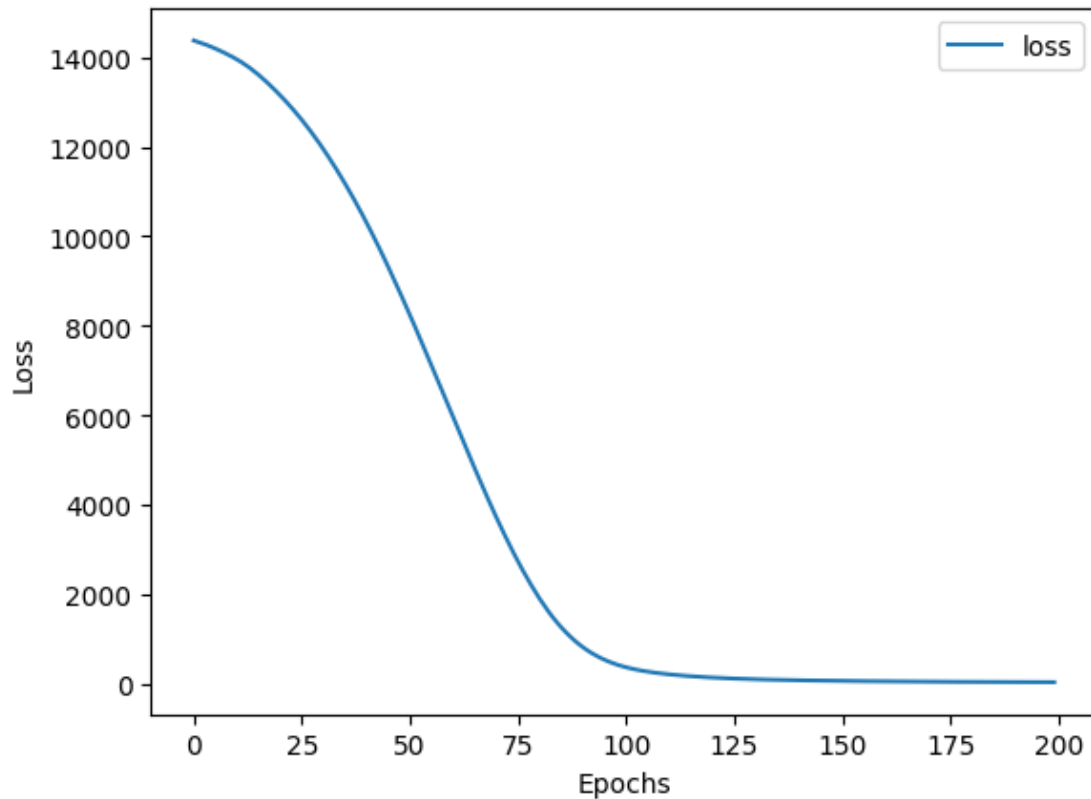
Evaluate Model

```
[44]: prob3_y_pred = prob3_model.predict(prob3_X_test_scaled)
```

```
1/1          0s 247ms/step
```

```
[48]: pd.DataFrame(history3.history).plot()
      mpl.pyplot.xlabel("Epochs")
      mpl.pyplot.ylabel("Loss");

```



```
[46]: mae = mean_absolute_error(prob3_y_test, prob3_y_pred)
      rmse = np.sqrt(mean_squared_error(prob3_y_test, prob3_y_pred))
      r2 = r2_score(prob3_y_test, prob3_y_pred)

      print(f"MAE: {mae:.4f}")
      print(f"RMSE: {rmse:.4f}")
      print(f"R²: {r2:.4f}")
```

MAE: 5.0203
 RMSE: 6.0414
 R²: 0.9927

```
[47]: comparison = pd.DataFrame({
      'Actual': prob3_y_test,
      'Predicted': prob3_y_pred.flatten(),
      'Error': prob3_y_test - prob3_y_pred.flatten()
    })
      print(comparison)
```

	Actual	Predicted	Error
30	32	36.763462	-4.763462
31	200	195.767731	4.232269

32	118	116.054604	1.945396
33	24	33.337246	-9.337246
34	62	59.605873	2.394127
35	225	226.516022	-1.516022
36	41	33.684948	7.315052
37	88	83.175720	4.824280
38	180	181.821274	-1.821274
39	55	42.946461	12.053539